Lecture 7: Efficient Querying

Indexes

- Data Structure for speeding up certain queries
- Especially useful for frequent attributes (After WHERE in SQL query)
- Without index for lastname - iterate over all n rows of table
  - -> runtime $\mathcal{O}(n)$ for each query
- Storing rows sorted by lastname would enable binary search:
  - -> runtime $\mathcal{O}(\log n)$ for a single query
- Need for *dynamic* sets to allow efficient queries and updates
- Often, balanced search trees
- E.g.: AVL-trees, Red-black-trees, B-trees
- Insert, Find, Delete need $\mathcal{O}(\log n)$ time
- **Syntax**:

```
CREATE INDEX idx ON customer USING btree(lastname);
```

Example

- table test with 10 mio rows
- attributes: id, random (random number in ([0,1])

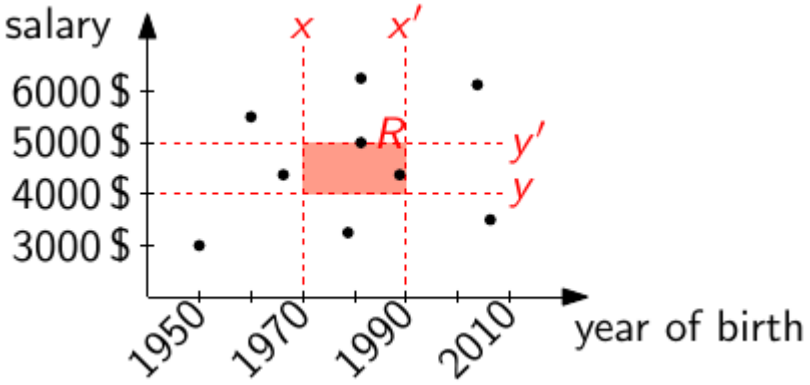
```
SELECT * FROM test WHERE random > 0.39999 AND random < 0.4;
```

- without index: 2070ms
- with index: 10ms

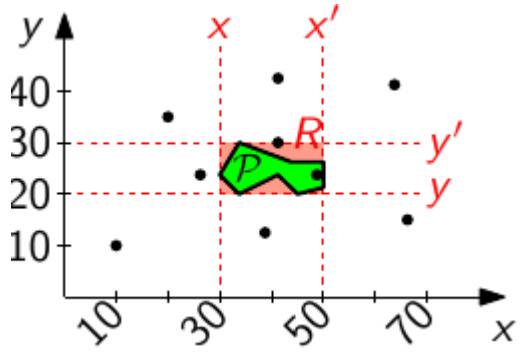
```
CREATE INDEX test_idx ON test USING btree (random);
```

Orthogonal Range Queries

- **Example**: Table "Employees" with columns "salary" (int) and "yearOfBirth" (int)
  - Each row is a point in $\mathbb{R}^2$

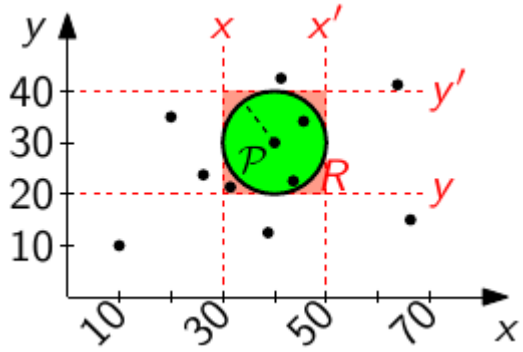


- Which employees were born between $x = 1970$ and $x' = 1990$ And salary between $y = 4000$ and $y' = 5000$?

- Find al point in rectangle $R = [x, x'] \times [y, y']$
- **most relevant for spatial databases**: x- and y-dimension are coordinates of points in Euclidean plane.
- Find all points within a given axis-aligned rectangle
  - 

- **frequent application**: Find points wihtin polygon
- **solution**: Execute range-query with bounding box R of $\mathcal{P}$
  - Then, for each returned $p \in R$, test whether $p \in \mathcal{P}$

Example

- Given set S of point sin the plane and $\epsilon > 0$:
  - Find all pairs of points $p, q \in S$ with $||p - q|| \leq \epsilon$
  - 

- **Create Index**:

```
CREATE INDEX lidarpoints_spx ON lidarpoints USING gist (geom);
```

- **Find all adjacent pairs**:

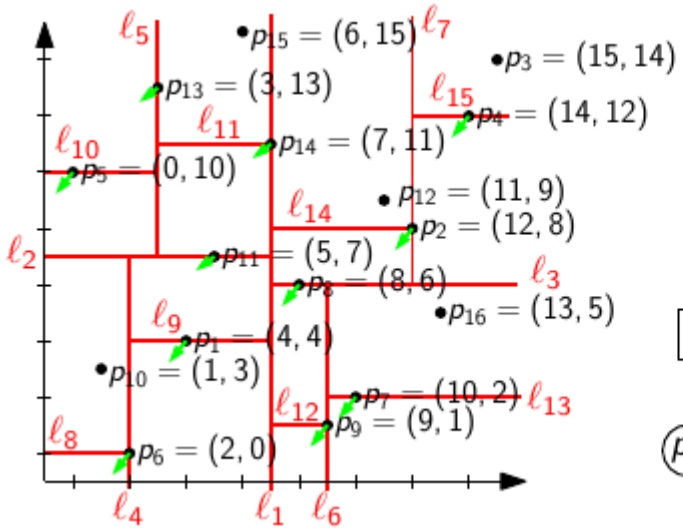
```
CREATE TABLE adjacencies AS
SELECT lp1.id AS source, lp2.id AS target
FROM lidarpoints AS lp1, lidarpoints AS lp2
WHERE ST_DWITHIN(lp1.geom, lp2.geom, 1) AND lp1.id<>lp2.id;
```

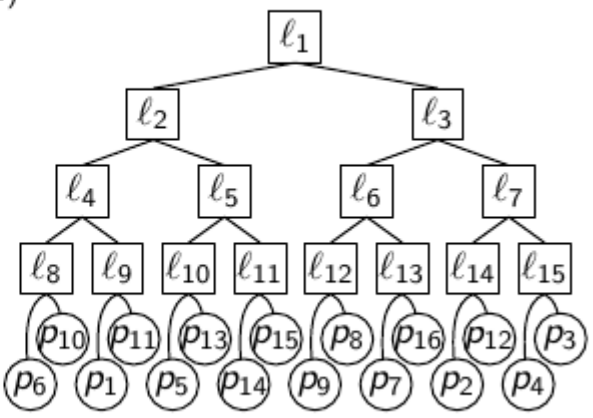
- `ST_DWITHIN` - returns true for points that are close enough and use rectangle query for that test
  - execution time: 1.8s
  - without index: 132s

kd-Trees

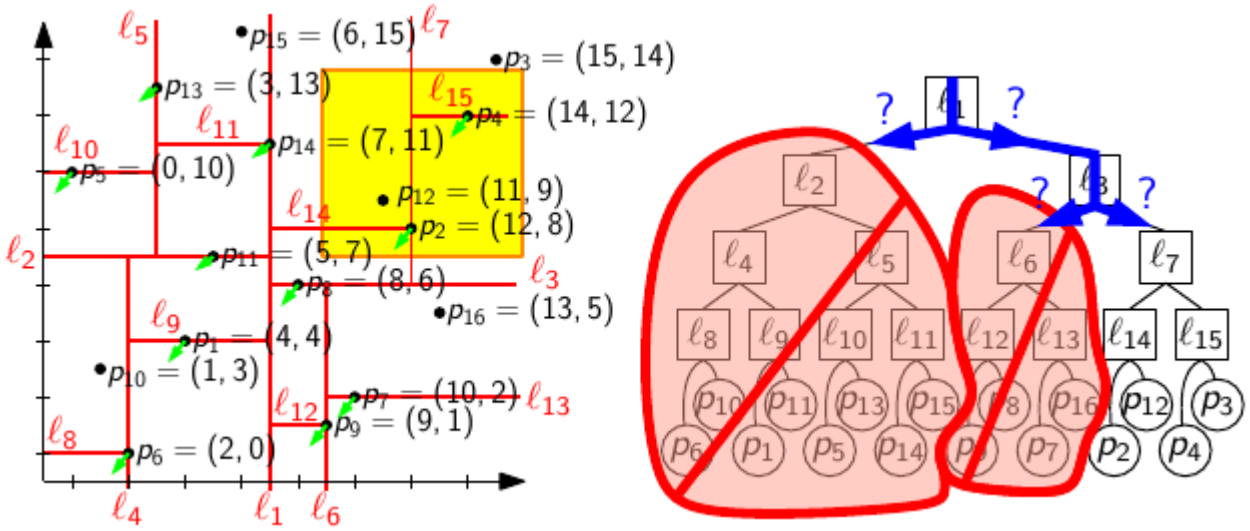
Basics

- **Assumptions**: (only for simplification of presentation)
  - *x-coordinates* of points are pair-wise distinct - same for *y-coordinates*
  - number n of points is a power of 4,
    - i.e. $n \in \{1, 4, 16, 64, ...\} \rightarrow n \in \{p | p = 2^{2n}, n \in \mathbb{N}^*\}$





- Vertical line containing point with median-x-coordinate:
  - tree root l_1
  - Set split by l_1 into 2 parts
- recursively partition both parts with horizontal lines, each at median-y-coordinate
- Until each part contains exactly one point (= leaf of tree)



- **Querying principle:**
  - Querying rectangle is completely right of l_1 and above l_3, -> left branch neglectible
- **Size in memory**: $n + \frac{n}{2} + \frac{n}{4} + ... + 1 < 2n \in \mathcal{O}(n)$

Construction of kd-Tree

Input: Point set P; current depth of tree ($depth$, initially 0)
Output: Root of tree.

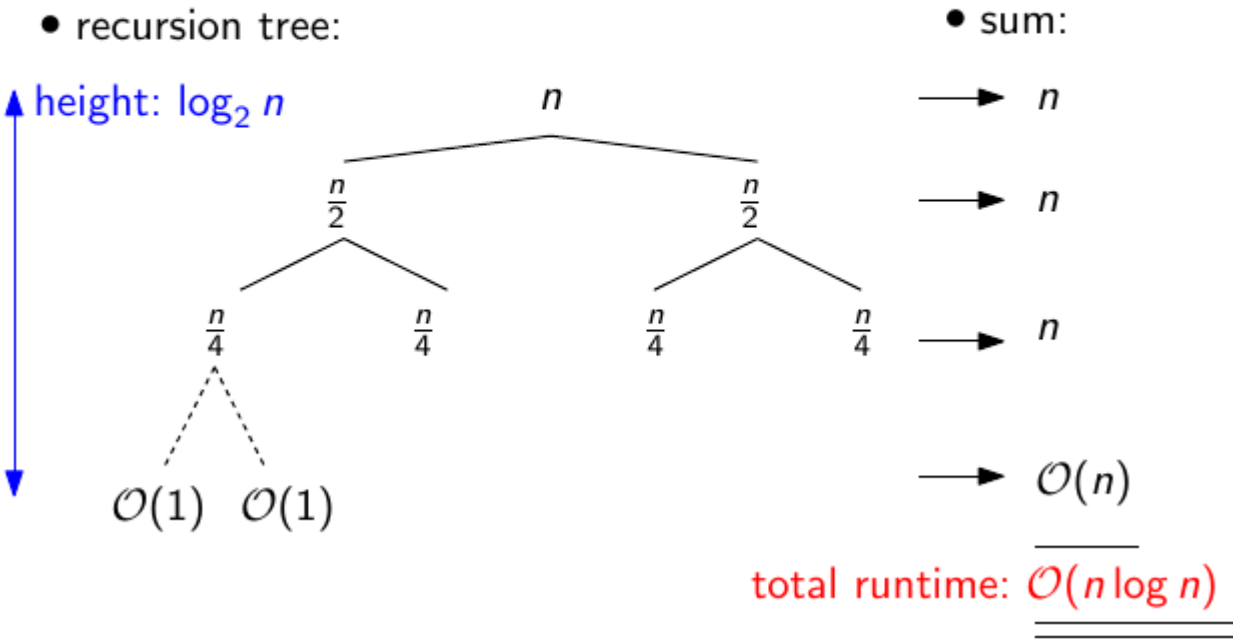
BuildKDTree(P, $depth$)

runtimes:

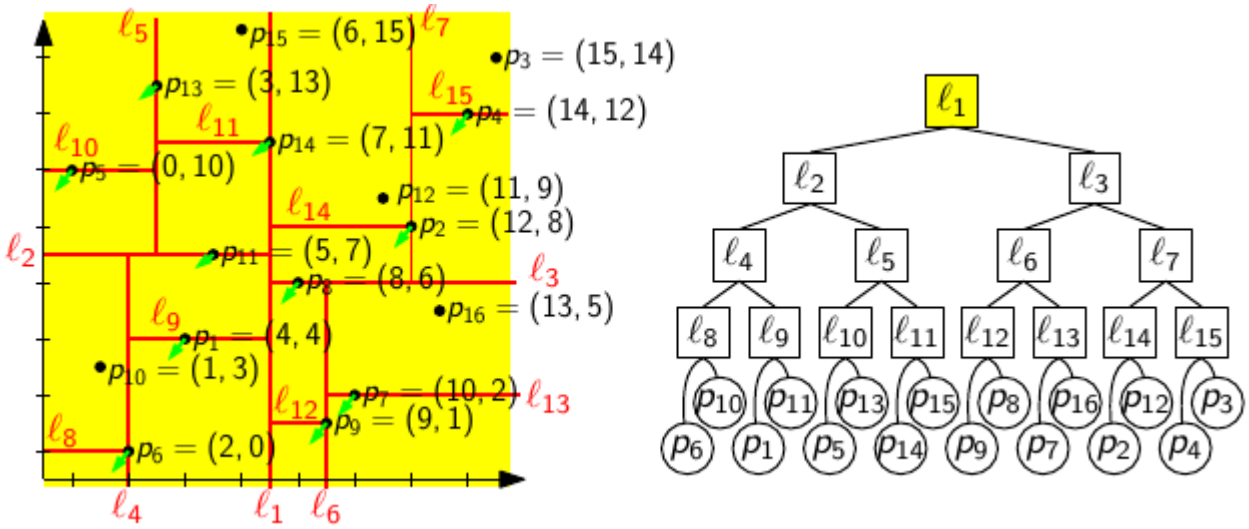
if $|P| == 1$ **then**
  return a leaf with a reference to the sole point in P — $\mathcal{O}(1)$
else
  if $depth$ is even **then** — $\mathcal{O}(|P|)$*
    Let ℓ be the vertical line containing the median-x-coord. of P.
    Partition P into subsets P_1 left and P_2 right of ℓ.
  else
    Let ℓ be the horizontal line containing the median-y-coord. of P.
    Partition P into subsets P_1 below and P_2 above ℓ.
  Let v be a new tree node with reference to ℓ.
  $v.leftChild =$ BuildKDTree(P_1, $depth + 1$) — $2T(|P|/2)$
  $v.rightChild =$ BuildKDTree(P_2, $depth + 1$)
  return v

$T(n) :=$ runtime of BuildKDTree for n points

$$\text{runtime } T(n) = \begin{cases} \mathcal{O}(1) & \text{if } n = 1 \\ 2T(n/2) + \mathcal{O}(n) & \text{if } n > 1 \end{cases}$$



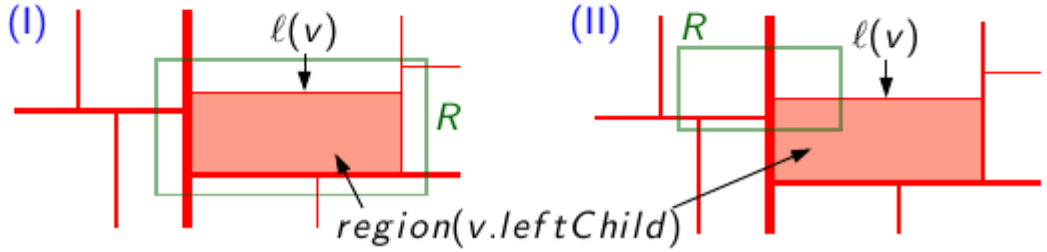
Querying a kd-Tree



Defintion:

- Each node v associated with region $region(v)$
- $Region(v)$ area enclosed by lines containing point corresponding to v
- For each other node v:
  - $region(v) = region(v.leftChild) \cup region(v.rightChild)$
- **Given**: root of v of (sub-)tree; query rectangle R
- **Output**: All points in R in subtree with root v

```
SearchKDTree(v, R)
  if v is a leaf then
    Report point p for v if p is in R.
  else
    if region(v.leftChild) is completely contained in R then
      Report all points in subtree with root v.leftChild        (I)
    else
      if region(v.leftChild) intersects R then                  (II)
        SearchKDTree(v.leftChild, R)
```




```
SearchKDTree(v, R)
  if v is a leaf then
    Report point p for v if p is in R.
  else
    if region(v.leftChild) is completely contained in R then
      Report all points in subtree with root v.leftChild
    else
      if region(v.leftChild) intersects R then
        SearchKDTree(v.leftChild, R)
    if region(v.rightChild) is completely contained in R then
      Report all points in subtree with root v.rightChild
    else
      if region(v.rightChild) intersects R then
        SearchKDTree(v.rightChild, R)
```

Summary

- kd-tree is an index structure for D-dimensional orthogonal range queries.
- Studied case $D = 2$.
- 2-dimensional kd-tree for n points
  - needs $\mathcal{O}(n)$ space in memory
  - can be computed in $\mathcal{O}(n \log n) time$
  - answers orthogonal range queries in $\mathcal{O}(\sqrt{n} + k)$
    - k is number of output points